CPU Scheduling: Worked Numericals

Chapter 5 - Operating Systems | Examples from the lecture slides, plus practice problems with answer key

Before You Start: Key Formulas

Every numerical in this handout uses the same five quantities. Compute them in this order for each process:

Quantity	Formula	Meaning
Completion Time (CT)	Time at which the process finishes	Read from the Gantt chart
Turnaround Time (TAT)	CT - Arrival Time (AT)	Total time spent in the system
Waiting Time (WT)	TAT - Burst Time (BT)	Time spent in the ready queue
Response Time (RT)	First time on CPU - AT	Time until first response
Throughput	No. of processes / total time	Processes completed per time unit

> **Conventions used**
> If no arrival time is given, all processes arrive at t = 0. A smaller priority number means higher priority (as on the slides). In Round Robin, if a new process arrives at the same instant a quantum expires, the new arrival joins the queue first. Ties are broken by arrival order, then by process number.

1. First-Come, First-Served (FCFS)

Idea: The process that arrives first gets the CPU first and keeps it until it finishes (non-preemptive). Simple, but a long process at the front delays everyone behind it (the **convoy effect**).

Example 1.1 - Arrival order P1, P2, P3 (Slide 12)

Process	Arrival Time	Burst Time
P1	0	24
P2	0	3
P3	0	3

Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	24	24 - 0 = 24	24 - 24 = 0	0 - 0 = 0
P2	27	27 - 0 = 27	27 - 3 = 24	24 - 0 = 24
P3	30	30 - 0 = 30	30 - 3 = 27	27 - 0 = 27
Average		**(24+27+30)/3 = 27**	**(0+24+27)/3 = 17**	**(0+24+27)/3 = 17**

Check against the slide: Waiting times are 0, 24, 27, so the average waiting time = (0 + 24 + 27)/3 = **17**.

Example 1.2 - Same processes, arrival order P2, P3, P1 (Slide 13)

Process	Arrival Time	Burst Time
P2	0	3
P3	0	3
P1	0	24

Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P2	3	3 - 0 = 3	3 - 3 = 0	0 - 0 = 0
P3	6	6 - 0 = 6	6 - 3 = 3	3 - 0 = 3
P1	30	30 - 0 = 30	30 - 24 = 6	6 - 0 = 6
Average		**(3+6+30)/3 = 13**	**(0+3+6)/3 = 3**	**(0+3+6)/3 = 3**

Check against the slide: Waiting times are P1 = 6, P2 = 0, P3 = 3, so the average waiting time = **3**.

> **Teaching point**
> The processes and burst times are identical, yet the average waiting time drops from 17 to 3 just by changing the order. This is the convoy effect: the two short jobs in Example 1.1 were stuck behind the 24-unit job.

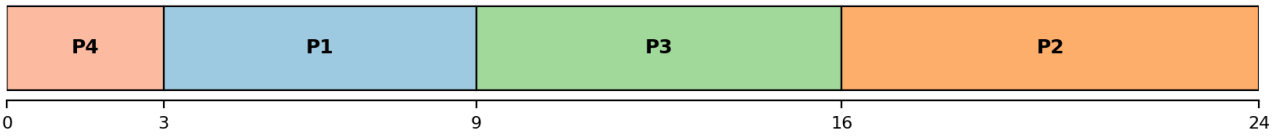
2. Shortest-Job-First (SJF, Non-Preemptive)

Idea: Among the processes that have arrived, pick the one with the smallest next CPU burst and let it run to completion. SJF gives the **minimum average waiting time** for a given set of processes.

Example 2.1 - All processes arrive at t = 0 (Slide 15)

Process	Arrival Time	Burst Time
P1	0	6
P2	0	8
P3	0	7
P4	0	3

Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	9	9 - 0 = 9	9 - 6 = 3	3 - 0 = 3
P2	24	24 - 0 = 24	24 - 8 = 16	16 - 0 = 16
P3	16	16 - 0 = 16	16 - 7 = 9	9 - 0 = 9
P4	3	3 - 0 = 3	3 - 3 = 0	0 - 0 = 0
Average		**(9+24+16+3)/4 = 13**	**(3+16+9+0)/4 = 7**	**(3+16+9+0)/4 = 7**

Check against the slide: Waiting times are 3, 16, 9, 0, so the average waiting time = (3 + 16 + 9 + 0)/4 = **7**.

Same data under FCFS, for comparison

FCFS order P1, P2, P3, P4 gives waiting times 0, 6, 14, 21, so average waiting time = **10.25**, compared to 7 under SJF.

3. Predicting the Next CPU Burst (Exponential Averaging)

SJF needs the length of the next CPU burst, which the OS cannot know in advance, so it is estimated from history (Slides 16-18):

> **Formula**
> tau(n+1) = alpha x t(n) + (1 - alpha) x tau(n)
> t(n) = actual length of the n-th CPU burst; tau(n) = predicted value for it; 0 <= alpha <= 1 (commonly 1/2).

Example 3.1 - alpha = 0.5, initial guess tau0 = 10

n	Actual burst t(n)	Predicted tau(n)	Next prediction tau(n+1)
0	6	10	0.5 x 6 + 0.5 x 10 = 8
1	4	8	0.5 x 4 + 0.5 x 8 = 6
2	6	6	0.5 x 6 + 0.5 x 6 = 6
3	4	6	0.5 x 4 + 0.5 x 6 = 5
4	13	5	0.5 x 13 + 0.5 x 5 = 9
5	13	9	0.5 x 13 + 0.5 x 9 = 11
6	13	11	0.5 x 13 + 0.5 x 11 = 12

Notice how the prediction chases the actual bursts: after three bursts of 13 it has risen from 5 to 12.

Example 3.2 - The two extreme values of alpha (Slide 18)

Take tau(n) = 10 and the latest actual burst t(n) = 6.

alpha	Calculation	Meaning
0	tau(n+1) = 0 x 6 + 1 x 10 = 10	Recent history does not count
1	tau(n+1) = 1 x 6 + 0 x 10 = 6	Only the last actual burst counts
0.5	tau(n+1) = 0.5 x 6 + 0.5 x 10 = 8	Equal weight to recent and past

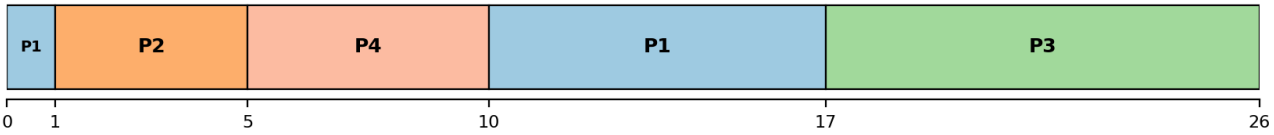
4. Shortest-Remaining-Time-First (SRTF, Preemptive SJF)

Idea: Whenever a new process arrives, compare its burst with the **remaining** time of the running process. If the newcomer is shorter, preempt.

Example 4.1 - With arrival times (Slide 20)

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	17	17 - 0 = 17	17 - 8 = 9	0 - 0 = 0
P2	5	5 - 1 = 4	4 - 4 = 0	1 - 1 = 0
P3	26	26 - 2 = 24	24 - 9 = 15	17 - 2 = 15
P4	10	10 - 3 = 7	7 - 5 = 2	5 - 3 = 2
Average		**(17+4+24+7)/4 = 13**	**(9+0+15+2)/4 = 6.5**	**(0+0+15+2)/4 = 4.25**

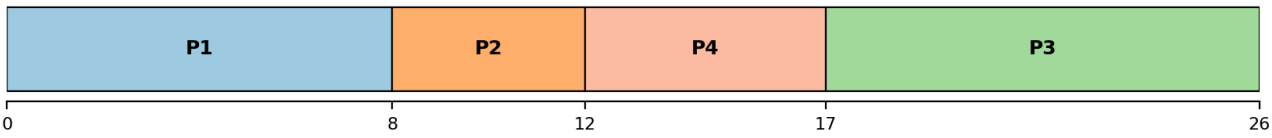
Step-by-step reasoning:

- t = 0: only P1 has arrived, so P1 runs.
- t = 1: P2 arrives with burst 4, while P1 has 7 remaining. 4 < 7, so P1 is preempted and P2 runs.
- t = 2 and 3: P3 (9) and P4 (5) arrive, but P2 has only 3 and then 2 units left, so P2 continues.
- t = 5: P2 finishes. Remaining times: P1 = 7, P3 = 9, P4 = 5. P4 is the shortest and runs.
- t = 10: P4 finishes. P1 (7) runs before P3 (9). At t = 17, P3 runs to completion at t = 26.

Check against the slide: Average waiting time = [(10-1) + (1-1) + (17-2) + (5-3)]/4 = 26/4 = **6.5**.

Is SRTF better than non-preemptive SJF? (Question on Slide 19)

Running the same four processes with non-preemptive SJF gives waiting times 0, 7, 15, 9, so the average waiting time = **7.75**. SRTF achieves **6.5**. When processes arrive at different times, SRTF gives the lower average waiting time, because it can react to a short job arriving while a long one is running.



(Gantt chart above: non-preemptive SJF on the same data.)

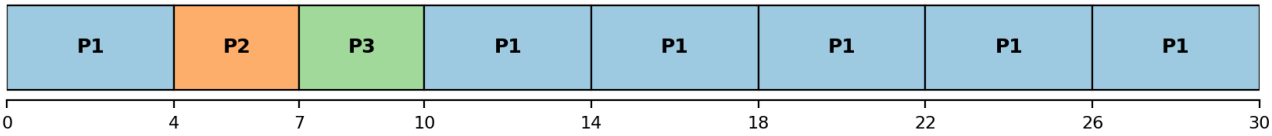
5. Round Robin (RR)

Idea: Each process gets the CPU for at most one time quantum q. If it has not finished, it goes to the back of the ready queue. With n processes, no process waits more than (n - 1) x q before getting the CPU.

Example 5.1 - Time quantum q = 4 (Slide 22)

Process	Arrival Time	Burst Time
P1	0	24
P2	0	3
P3	0	3

Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	30	30 - 0 = 30	30 - 24 = 6	0 - 0 = 0
P2	7	7 - 0 = 7	7 - 3 = 4	4 - 0 = 4
P3	10	10 - 0 = 10	10 - 3 = 7	7 - 0 = 7
Average		**(30+7+10)/3 = 15.67**	**(6+4+7)/3 = 5.67**	**(0+4+7)/3 = 3.67**

How to read it: P1 runs 0-4 (20 left), P2 needs only 3 so it finishes at 7, P3 finishes at 10, then P1 is the only process left and runs its remaining 20 units in chunks of 4 until t = 30.

Check against the slide: P1 waits 10 - 4 = 6, P2 waits 4, P3 waits 7, so average waiting time = 17/3 = **5.67**. Compare with FCFS (17) on the same data: RR is far better here and gives a quick first response to every process.

Example 5.2 - Effect of the time quantum (same processes)

Quantum q	Avg waiting	Avg turnaround	Avg response
1	5.67	15.67	1
2	6.33	16.33	2
4	5.67	15.67	3.67
6	7	17	5
10	9.67	19.67	7.67
30	17	27	17

As q grows beyond the longest burst, RR behaves exactly like FCFS (q = 30 row: average waiting time 17). As q shrinks, response time improves but context switches increase.

Example 5.3 - Context-switch overhead (Slide 23)

A process needs a 10-unit CPU burst. How many context switches occur for different quanta?

Quantum q	Context switches	Reasoning
12	0	Burst fits inside one quantum, so the process finishes without being preempted
6	1	Runs 0-6, preempted once, finishes 6-10
1	9	Preempted after every unit except the last: 10 - 1 = 9 switches

Example 5.4 - Maximum waiting time and CPU efficiency

- **Maximum wait:** 5 processes in the ready queue, q = 20 ms. A process waits at most (n - 1) x q = (5 - 1) x 20 = **80 ms** before its next turn.
- **Efficiency:** q = 20 ms and a context switch costs 2 ms. Useful fraction = q/(q + switch) = 20/22 = **90.9%**, so about 9.1% of CPU time is overhead. With q = 2 ms the efficiency would fall to 2/4 = 50%, which is why q must be large compared to the context-switch time.

6. Priority Scheduling

Idea: Each process has a priority number; the CPU goes to the highest priority (smallest integer = highest priority). It can be non-preemptive or preemptive. Problem: **starvation** of low-priority processes. Solution: **aging**.

Example 6.1 - Non-preemptive, all arrive at t = 0 (Slide 26)

Process	Arrival Time	Burst Time	Priority
P1	0	10	3
P2	0	1	1
P3	0	2	4
P4	0	1	5
P5	0	5	2

Gantt chart:



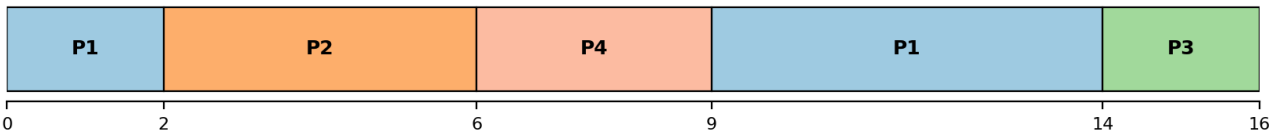
Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	16	16 - 0 = 16	16 - 10 = 6	6 - 0 = 6
P2	1	1 - 0 = 1	1 - 1 = 0	0 - 0 = 0
P3	18	18 - 0 = 18	18 - 2 = 16	16 - 0 = 16
P4	19	19 - 0 = 19	19 - 1 = 18	18 - 0 = 18
P5	6	6 - 0 = 6	6 - 5 = 1	1 - 0 = 1
Average		**(16+1+18+19+6)/5 = 12**	**(6+0+16+18+1)/5 = 8.2**	**(6+0+16+18+1)/5 = 8.2**

Check against the slide: Waiting times are P1 = 6, P2 = 0, P3 = 16, P4 = 18, P5 = 1, so average waiting time = 41/5 = **8.2**.

Example 6.2 - Preemptive priority with arrival times

Process	Arrival Time	Burst Time	Priority
P1	0	7	3
P2	2	4	1
P3	4	2	4
P4	5	3	2

Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	14	14 - 0 = 14	14 - 7 = 7	0 - 0 = 0
P2	6	6 - 2 = 4	4 - 4 = 0	2 - 2 = 0
P3	16	16 - 4 = 12	12 - 2 = 10	14 - 4 = 10
P4	9	9 - 5 = 4	4 - 3 = 1	6 - 5 = 1
Average		**(14+4+12+4)/4 = 8.5**	**(7+0+10+1)/4 = 4.5**	**(0+0+10+1)/4 = 2.75**

Reasoning: P1 starts alone. At t = 2, P2 arrives with priority 1 (better than 3), so P1 is preempted. P2 finishes at 6. P4 (priority 2) then beats P1 (3) and P3 (4); finally P1 and P3 follow in priority order.

Example 6.3 - Aging

A system uses priority numbers 0-127 (0 = highest). A process is waiting at priority 127. The scheduler **decrements its priority number by 1 every 15 minutes** of waiting. How long until it reaches the highest priority (0)?

- Steps needed = 127 - 0 = 127

- Time = 127 x 15 min = 1905 min = **31.75 hours** (about 1 day 8 hours)

Without aging, if higher-priority processes keep arriving, this process could wait forever. Aging guarantees that it eventually runs.

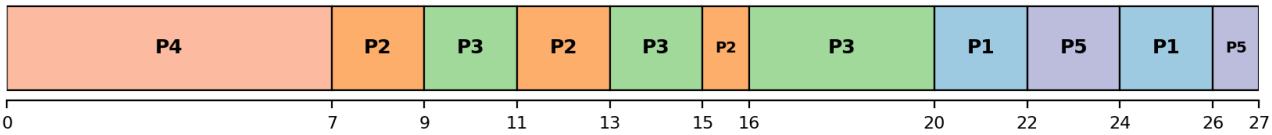
7. Priority Scheduling with Round Robin

Idea: Always run the highest-priority processes. Processes that share the **same** priority take turns in Round Robin.

Example 7.1 - Time quantum q = 2 (Slide 27)

Process	Arrival Time	Burst Time	Priority
P1	0	4	3
P2	0	5	2
P3	0	8	2
P4	0	7	1
P5	0	3	3

Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	26	26 - 0 = 26	26 - 4 = 22	20 - 0 = 20
P2	16	16 - 0 = 16	16 - 5 = 11	7 - 0 = 7
P3	20	20 - 0 = 20	20 - 8 = 12	9 - 0 = 9
P4	7	7 - 0 = 7	7 - 7 = 0	0 - 0 = 0
P5	27	27 - 0 = 27	27 - 3 = 24	22 - 0 = 22
Average		**(26+16+20+7+27)/5 = 19.2**	**(22+11+12+0+24)/5 = 13.8**	**(20+7+9+0+22)/5 = 11.6**

Reasoning by priority level:

- Priority 1: P4 is alone, so it runs 0-7.
- Priority 2: P2 and P3 share the CPU in RR (q = 2): P2, P3, P2, P3, P2, P3, P3 from t = 7 to t = 20. P2 finishes at 16, P3 at 20.
- Priority 3: P1 and P5 alternate in RR from t = 20: P1, P5, P1, P5, finishing at t = 26 (P1) and t = 27 (P5).

8. Multilevel Queue Scheduling

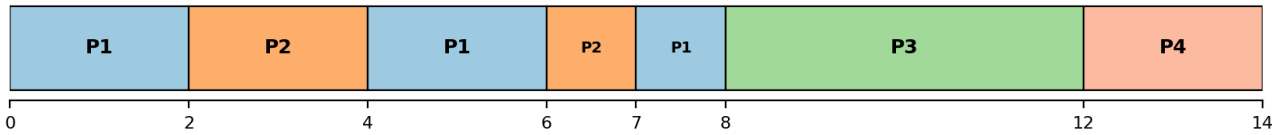
Idea: The ready queue is split into separate queues (for example foreground/interactive and background/batch). Each queue has its own algorithm, and the queues are scheduled against each other with **fixed priority**: the background queue runs only when the foreground queue is empty.

Example 8.1 - Two queues

> **Setup**
> Queue 1 (foreground, higher priority): Round Robin, q = 2 | Queue 2 (background): FCFS

Process	Queue	Arrival	Burst
P1	Foreground (RR)	0	5
P2	Foreground (RR)	0	3
P3	Background (FCFS)	0	4
P4	Background (FCFS)	0	2

Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	8	8 - 0 = 8	8 - 5 = 3	0 - 0 = 0
P2	7	7 - 0 = 7	7 - 3 = 4	2 - 0 = 2
P3	12	12 - 0 = 12	12 - 4 = 8	8 - 0 = 8
P4	14	14 - 0 = 14	14 - 2 = 12	12 - 0 = 12
Average		**(8+7+12+14)/4 = 10.25**	**(3+4+8+12)/4 = 6.75**	**(0+2+8+12)/4 = 5.5**

Reasoning: The foreground queue has absolute priority, so P1 and P2 alternate (q = 2) until both finish at t = 8. Only then does P3 get the CPU (8-12), followed by P4 (12-14). Notice the background processes have large waiting times; this is the price of strict priority between queues.

9. Multilevel Feedback Queue (MLFQ)

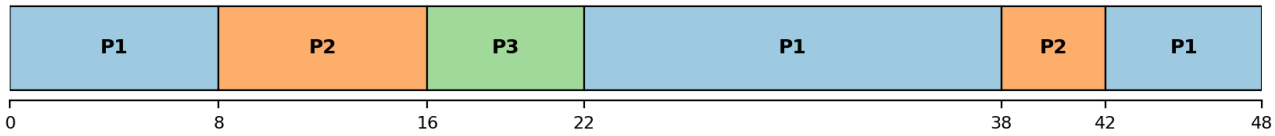
Idea: Like multilevel queue, but processes **move between queues**. A process that uses its full quantum is demoted; this keeps short, interactive jobs in the upper queues and pushes CPU-hungry jobs down.

Example 9.1 - The three-queue configuration from Slide 32

> **Setup**
> Q0: RR, q = 8 ms | Q1: RR, q = 16 ms | Q2: FCFS
> A new process enters Q0. If it does not finish within its quantum, it is moved down one queue. A lower queue runs only when all higher queues are empty.

Process	Arrival Time	Burst Time
P1	0	30
P2	0	12
P3	0	6

Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	48	48 - 0 = 48	48 - 30 = 18	0 - 0 = 0
P2	42	42 - 0 = 42	42 - 12 = 30	8 - 0 = 8
P3	22	22 - 0 = 22	22 - 6 = 16	16 - 0 = 16
Average		**(48+42+22)/3 = 37.33**	**(18+30+16)/3 = 21.33**	**(0+8+16)/3 = 8**

Trace:

- **Q0 (q = 8):** P1 runs 0-8 (22 left, demoted to Q1). P2 runs 8-16 (4 left, demoted to Q1). P3 needs only 6, so it finishes at 22.
- **Q1 (q = 16):** P1 runs 22-38 (6 left, demoted to Q2). P2 needs only 4, so it finishes at 42.
- **Q2 (FCFS):** P1 runs its last 6 units, 42-48.

The short job P3 finishes first and the longest job P1 finishes last, which is the intended behavior of MLFQ.

Practice Problems

Solve each problem by drawing the Gantt chart and computing CT, TAT, WT and RT for every process. Answers with Gantt charts follow in the Answer Key.

Problem 1 - Compare four algorithms

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Find the average waiting time and average turnaround time using (a) FCFS, (b) non-preemptive SJF, (c) SRTF, (d) Round Robin with q = 3. Which algorithm gives the smallest average waiting time?

Problem 2 - Convoy effect

Process	Burst Time
P1	30
P2	4
P3	2

(a) Compute the average waiting time under FCFS with arrival order P1, P2, P3. (b) Compute it again when the order is P3, P2, P1. (c) What is the percentage improvement?

Problem 3 - Exponential averaging

A process has had CPU bursts of 5, 9, 4 and 12 time units (in that order). The initial estimate is tau0 = 8 and alpha = 0.5. (a) Predict the length of the next burst after each actual burst. (b) Repeat with alpha = 0.8. Which setting reacts faster to the jump to 12?

Problem 4 - Priority scheduling

Process	Arrival Time	Burst Time	Priority
P1	0	6	3
P2	1	4	1
P3	2	5	4
P4	3	2	2

Compute the average waiting time with (a) non-preemptive priority and (b) preemptive priority (smaller number = higher priority).

Problem 5 - Priority with Round Robin

Process	Burst Time	Priority
P1	5	2
P2	4	1
P3	3	2
P4	6	3

Using time quantum q = 2 for processes of equal priority, draw the Gantt chart and find the average waiting time.

Problem 6 - Multilevel queue

Queue 1 (higher priority) uses RR with q = 3 and holds P1 (burst 6) and P2 (burst 4). Queue 2 uses FCFS and holds P3 (burst 5) and P4 (burst 3). All arrive at t = 0. Draw the Gantt chart and find the average waiting time. Which processes suffer most, and why?

Problem 7 - Multilevel feedback queue

Q0: RR q = 4, Q1: RR q = 8, Q2: FCFS. A process is demoted when it uses its full quantum. All arrive at t = 0.

Process	Burst Time
P1	20
P2	6
P3	14

Draw the Gantt chart and compute the average turnaround time.

Problem 8 - Round Robin parameters

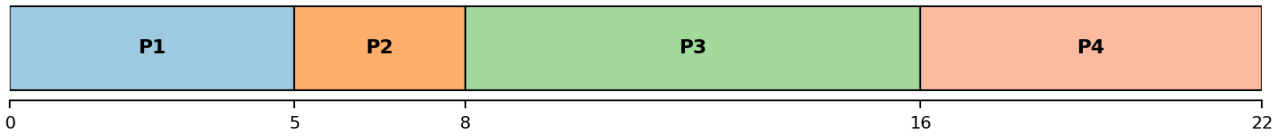
(a) There are 6 processes in the ready queue and q = 15 ms. What is the longest a process can wait for its next turn? (b) If a context switch takes 1 ms, what fraction of CPU time is useful when q = 9 ms? When q = 1 ms?

Answer Key

Problem 1

(a) FCFS

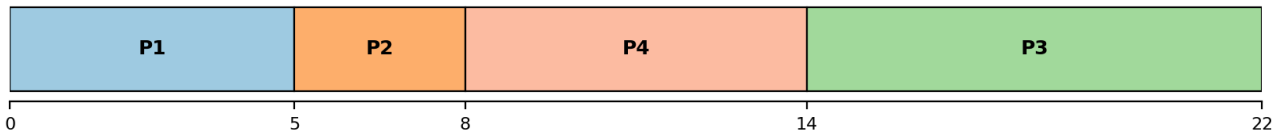
Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	5	5 - 0 = 5	5 - 5 = 0	0 - 0 = 0
P2	8	8 - 1 = 7	7 - 3 = 4	5 - 1 = 4
P3	16	16 - 2 = 14	14 - 8 = 6	8 - 2 = 6
P4	22	22 - 3 = 19	19 - 6 = 13	16 - 3 = 13
Average		**(5+7+14+19)/4 = 11.25**	**(0+4+6+13)/4 = 5.75**	**(0+4+6+13)/4 = 5.75**

(b) SJF (non-preemptive)

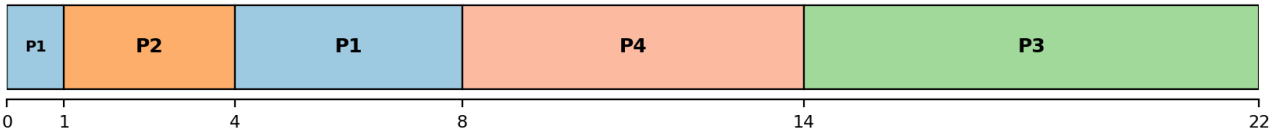
Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	5	5 - 0 = 5	5 - 5 = 0	0 - 0 = 0
P2	8	8 - 1 = 7	7 - 3 = 4	5 - 1 = 4
P3	22	22 - 2 = 20	20 - 8 = 12	14 - 2 = 12
P4	14	14 - 3 = 11	11 - 6 = 5	8 - 3 = 5
Average		**(5+7+20+11)/4 = 10.75**	**(0+4+12+5)/4 = 5.25**	**(0+4+12+5)/4 = 5.25**

(c) SRTF

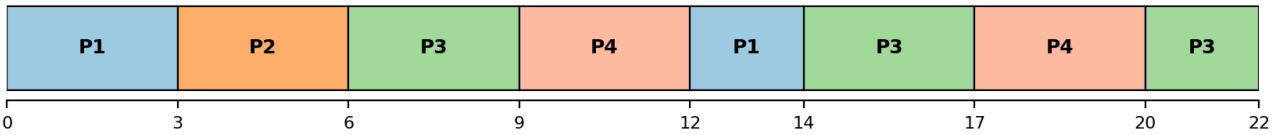
Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	8	8 - 0 = 8	8 - 5 = 3	0 - 0 = 0
P2	4	4 - 1 = 3	3 - 3 = 0	1 - 1 = 0
P3	22	22 - 2 = 20	20 - 8 = 12	14 - 2 = 12
P4	14	14 - 3 = 11	11 - 6 = 5	8 - 3 = 5
Average		**(8+3+20+11)/4 = 10.5**	**(3+0+12+5)/4 = 5**	**(0+0+12+5)/4 = 4.25**

(d) Round Robin, q = 3

Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	14	14 - 0 = 14	14 - 5 = 9	0 - 0 = 0
P2	6	6 - 1 = 5	5 - 3 = 2	3 - 1 = 2
P3	22	22 - 2 = 20	20 - 8 = 12	6 - 2 = 4
P4	20	20 - 3 = 17	17 - 6 = 11	9 - 3 = 6
Average		**(14+5+20+17)/4 = 14**	**(9+2+12+11)/4 = 8.5**	**(0+2+4+6)/4 = 3**

Summary

Algorithm	Avg waiting	Avg turnaround	Avg response
FCFS	5.75	11.25	5.75
SJF	5.25	10.75	5.25
SRTF	5	10.5	4.25
RR (q = 3)	8.5	14	3

Smallest average waiting time: SRTF (5). Note that SRTF and SJF minimize waiting time but can have poor response for long jobs, while RR gives the best response at the cost of higher turnaround.

Problem 2

- (a) Order P1, P2, P3: waiting times P1 = 0, P2 = 30, P3 = 34, average = (0 + 30 + 34)/3 = **21.33**.
- (b) Order P3, P2, P1: waiting times P3 = 0, P2 = 2, P1 = 6, average = (0 + 2 + 6)/3 = **2.67**.
- (c) Improvement = (21.33 - 2.67)/21.33 x 100 = **87.5%**.

Problem 3

(a) alpha = 0.5

n	Actual t(n)	Predicted tau(n)	Next prediction
0	5	8	0.5 x 5 + 0.5 x 8 = 6.5
1	9	6.5	0.5 x 9 + 0.5 x 6.5 = 7.75
2	4	7.75	0.5 x 4 + 0.5 x 7.75 = 5.88
3	12	5.88	0.5 x 12 + 0.5 x 5.88 = 8.94

(b) alpha = 0.8

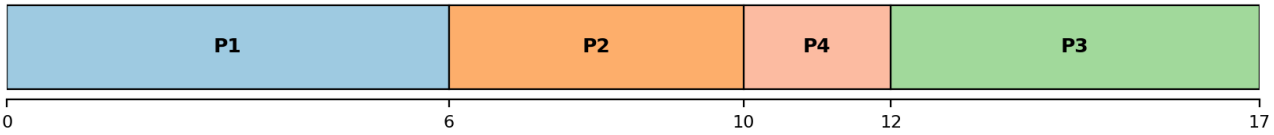
n	Actual t(n)	Predicted tau(n)	Next prediction
0	5	8	0.8 x 5 + 0.2 x 8 = 5.6
1	9	5.6	0.8 x 9 + 0.2 x 5.6 = 8.32
2	4	8.32	0.8 x 4 + 0.2 x 8.32 = 4.86
3	12	4.86	0.8 x 12 + 0.2 x 4.86 = 10.57

With alpha = 0.8 the final prediction is closer to the latest burst of 12, so the larger alpha reacts faster to recent changes.

Problem 4

(a) Non-preemptive priority

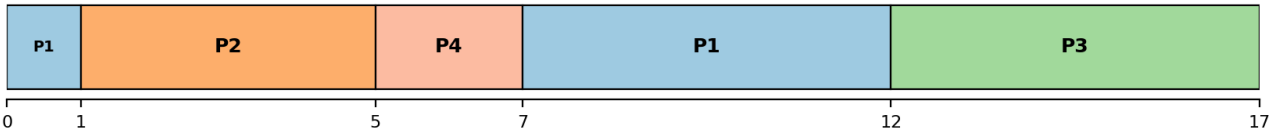
Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	6	6 - 0 = 6	6 - 6 = 0	0 - 0 = 0
P2	10	10 - 1 = 9	9 - 4 = 5	6 - 1 = 5
P3	17	17 - 2 = 15	15 - 5 = 10	12 - 2 = 10
P4	12	12 - 3 = 9	9 - 2 = 7	10 - 3 = 7
Average		**(6+9+15+9)/4 = 9.75**	**(0+5+10+7)/4 = 5.5**	**(0+5+10+7)/4 = 5.5**

(b) Preemptive priority

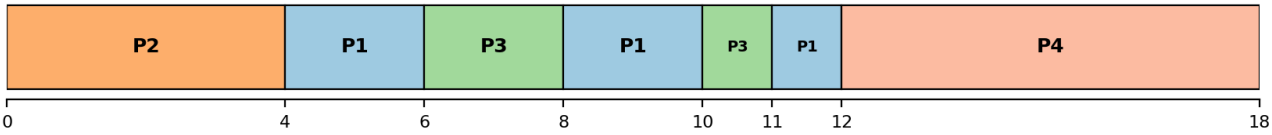
Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	12	12 - 0 = 12	12 - 6 = 6	0 - 0 = 0
P2	5	5 - 1 = 4	4 - 4 = 0	1 - 1 = 0
P3	17	17 - 2 = 15	15 - 5 = 10	12 - 2 = 10
P4	7	7 - 3 = 4	4 - 2 = 2	5 - 3 = 2
Average		**(12+4+15+4)/4 = 8.75**	**(6+0+10+2)/4 = 4.5**	**(0+0+10+2)/4 = 3**

Problem 5

Gantt chart:

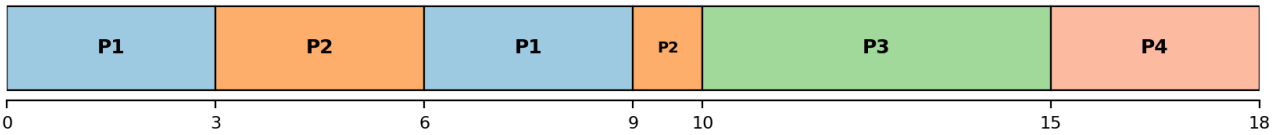


Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	12	12 - 0 = 12	12 - 5 = 7	4 - 0 = 4
P2	4	4 - 0 = 4	4 - 4 = 0	0 - 0 = 0
P3	11	11 - 0 = 11	11 - 3 = 8	6 - 0 = 6
P4	18	18 - 0 = 18	18 - 6 = 12	12 - 0 = 12
Average		**(12+4+11+18)/4 = 11.25**	**(7+0+8+12)/4 = 6.75**	**(4+0+6+12)/4 = 5.5**

P2 (priority 1) runs alone first. P1 and P3 (priority 2) then alternate with q = 2, and P4 (priority 3) runs last.

Problem 6

Gantt chart:

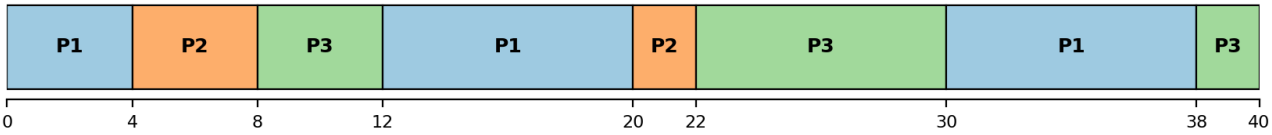


Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	9	9 - 0 = 9	9 - 6 = 3	0 - 0 = 0
P2	10	10 - 0 = 10	10 - 4 = 6	3 - 0 = 3
P3	15	15 - 0 = 15	15 - 5 = 10	10 - 0 = 10
P4	18	18 - 0 = 18	18 - 3 = 15	15 - 0 = 15
Average		**(9+10+15+18)/4 = 13**	**(3+6+10+15)/4 = 8.5**	**(0+3+10+15)/4 = 7**

P3 and P4 suffer most: they only run after the foreground queue is empty, so they wait for all foreground work (10 units) and, for P4, also for P3. This is the cost of fixed priority between queues, and why starvation is possible if foreground work keeps arriving.

Problem 7

Gantt chart:



Process	Completion (CT)	Turnaround = CT - AT	Waiting = TAT - BT	Response = First CPU - AT
P1	38	38 - 0 = 38	38 - 20 = 18	0 - 0 = 0
P2	22	22 - 0 = 22	22 - 6 = 16	4 - 0 = 4
P3	40	40 - 0 = 40	40 - 14 = 26	8 - 0 = 8
Average		**(38+22+40)/3 = 33.33**	**(18+16+26)/3 = 20**	**(0+4+8)/3 = 4**

P2 (6 units) finishes in Q1 after one demotion. P1 and P3 are demoted to Q2 and finish last, in FCFS order.

Problem 8

- (a) Maximum wait = (n - 1) x q = (6 - 1) x 15 = **75 ms**.
- (b) Useful fraction = q/(q + switch). For q = 9 ms: 9/10 = **90%**. For q = 1 ms: 1/2 = **50%**. A very small quantum wastes half of the CPU on context switches.

Quick Comparison of Algorithms

Algorithm	Preemptive?	Selection rule	Strength / weakness
FCFS	No	Earliest arrival	Simple; convoy effect gives long average waiting
SJF	No	Shortest next burst	Minimum average waiting time; needs burst prediction; can starve long jobs
SRTF	Yes	Shortest remaining time	Best average waiting with arrivals; frequent preemption
Round Robin	Yes	Time quantum q, circular queue	Good response; higher turnaround; q must exceed context-switch time
Priority	Either	Smallest priority number	Flexible; starvation fixed with aging
Priority + RR	Yes	Priority, then RR within a level	Fair among equals, strict among different priorities
Multilevel Queue	Yes	Fixed priority between queues	Different algorithms per process type; background can starve
Multilevel Feedback	Yes	Demote on full quantum; age to promote	Adaptive; favors short jobs; most complex to tune